Human Verification: `Containers.Network`

Shipping Networks as Directed Containers

Machine-generated from `lean/Network.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.Network` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

The file verifies Construction `con:network` (§4 of the paper): every directed graph G induces a directed container $\mathcal{N}(G)$ whose shapes are nodes, whose positions at a node v are finite directed paths out of v, and whose directed-container laws follow from path-concatenation identities. The file ends with the 11-facility Sawasdee Cellars wine network as the paper's running example.

Contents

1 Overview **1**

2 Directed graphs **2**
2.1 Structure: `Digraph` . . . 2
2.2 Inductive type: `Digraph.Path` . . . 2
2.3 Definition: `Path.append` . . . 3
2.4 Theorems: `Path.nil_append`, `Path.append_nil`, `Path.append_assoc` . . . 3

3 The network directed container **3**
3.1 Definition: `Digraph.network` . . . 3

4 The Sawasdee Cellars wine network **4**
4.1 Types and graph: `Wine.Facility`, `Wine.Leg`, `Wine.graph` . . . 4
4.2 Definition: `Wine.network` . . . 5
4.3 Sanity checks (examples checked by `rfl`) . . . 5

5 Translation conventions **6**

1 Overview

A *digraph* $G = (V, E, \mathrm{src}, \mathrm{tgt})$ gives rise to a directed container $\mathcal{N}(G)$ — the *network container* — following the recipe:

- **Shape** $= V$ (nodes);
- $\mathbf{Pos}(v) = \{(w, \rho) \mid \rho \text{ is a path } v \rightsquigarrow w\}$ (routes out of v, together with their endpoint);

- **root**$(v) = (v, \varepsilon)$ (the empty path);
- **sub**$(v, (w, \rho)) = w$ (the endpoint of the route);
- **shift**$(v, (w, \rho), (u, \sigma)) = (u, \rho +\!\!+ \sigma)$ (route concatenation).

As a category (via `DContCat.cat`) this is the free category on G.

2 Directed graphs

2.1 Structure: `Digraph`

Definition 2.1 (Directed graph)**.** A **directed (multi-)graph** G consists of a type V of vertices, a type E of edges, and two maps $\mathrm{src}, \mathrm{tgt} : E \to V$ giving the source and target of each edge. Parallel edges and loops are allowed; no finiteness is assumed.

Lean.
```
structure Digraph where
  V   : Type
  E   : Type
  src : E → V
  tgt : E → V
```

Haskell.
```
data Digraph v e = Digraph
  { src :: e → v
  , tgt :: e → v
  }
```

2.2 Inductive type: `Digraph.Path`

Definition 2.2 (Directed path)**.** A **directed path** $\rho : \mathrm{Path}_G(v, w)$ is defined inductively:

- $\mathrm{nil}(v) : \mathrm{Path}_G(v, v)$ — the empty path at v;
- $\mathrm{cons}(e, \rho) : \mathrm{Path}_G(\mathrm{src}(e), w)$ when $\rho : \mathrm{Path}_G(\mathrm{tgt}(e), w)$ — prepend an edge.

Paths are cons-at-the-front: the first constructor argument is an outgoing edge from the start node, followed by a path from its target.

Lean.
```
inductive Path (G : Digraph) : G.V → G.V → Type
  | nil  (v : G.V)                          : Path G v v
  | cons (e : G.E) {w : G.V}
         (p : Path G (G.tgt e) w)           : Path G (G.src e) w
```

Haskell.
```
-- Approximation: a path is a non-empty-or-empty list of edges.
-- The endpoint types v, w are not tracked.
data Path e = Nil | Cons e (Path e)
```

Fidelity note. In Lean, `Path G v w` is a *type* indexed by both endpoints; the type system enforces that consecutive edges match. In Haskell, this dependency is erased: a path is just a list of edges, and endpoint compatibility is a runtime property only.

2.3 Definition: `Path.append`

Definition 2.3 (Path concatenation)**.** Given paths $\rho : \mathrm{Path}_G(v, w)$ and $\sigma : \mathrm{Path}_G(w, u)$, their concatenation $\rho +\!\!+ \sigma : \mathrm{Path}_G(v, u)$ is defined by recursion on ρ:

$$\mathrm{nil}(v) +\!\!+ \sigma = \sigma, \qquad \mathrm{cons}(e, \rho) +\!\!+ \sigma = \mathrm{cons}(e,\ \rho +\!\!+ \sigma).$$

Lean.

```
def append : {v w u : G.V} → Path G v w → Path G w u → Path G v u
  | _, _, _, .nil _,     q ⇒ q
  | _, _, _, .cons e p, q ⇒ .cons e (p.append q)
```

Haskell.

```
append :: Path e → Path e → Path e
append Nil          q = q
append (Cons e p)   q = Cons e (append p q)
```

2.4 Theorems: `Path.nil_append`, `Path.append_nil`, `Path.append_assoc`

Theorem 2.4 (Path monoid laws)**.** For all paths $\rho : \mathrm{Path}_G(v, w)$, $\sigma : \mathrm{Path}_G(w, u)$, $\tau : \mathrm{Path}_G(u, x)$:

1. **Left unit**: $\mathrm{nil}(v) +\!\!+ \rho = \rho$ (definitional).
2. **Right unit**: $\rho +\!\!+ \mathrm{nil}(w) = \rho$.
3. **Associativity**: $(\rho +\!\!+ \sigma) +\!\!+ \tau = \rho +\!\!+ (\sigma +\!\!+ \tau)$.

Lean.

```
theorem nil_append {v w : G.V} (p : Path G v w) :
    (Path.nil v).append p = p

theorem append_nil : {v w : G.V} → (p : Path G v w) →
    p.append (Path.nil w) = p

theorem append_assoc : {v w u x : G.V} →
    (p : Path G v w) → (q : Path G w u) → (r : Path G u x) →
    (p.append q).append r = p.append (q.append r)
```

Haskell.

```
-- Left unit  (by definition):  append Nil q = q
-- Right unit (by induction):   append p Nil = p
-- Associativity (by induction):
--   append (append p q) r = append p (append q r)
```

3 The network directed container

3.1 Definition: `Digraph.network`

Definition 3.1 (Network container)**.** For a digraph G, the **network directed container** $\mathcal{N}(G)$ is the directed container with:

$$\begin{aligned} \mathrm{Shape} &= G.V, \\ \mathrm{Pos}(v) &= \textstyle\sum_{w : G.V} \mathrm{Path}_G(v, w), \\ \mathrm{root}(v) &= (v,\ \mathrm{nil}(v)), \\ \mathrm{sub}(v, (w, \rho)) &= w, \\ \mathrm{shift}(v, (w, \rho), (u, \sigma)) &= (u,\ \rho +\!\!+ \sigma). \end{aligned}$$

The directed-container laws D1, D2, D4 hold definitionally (`rfl`); D3 is the right-unit law and D5 is associativity of `append`.

Lean.

```
def network (G : Digraph) : DirectedContainer where
  Shape := G.V
  Pos v := (w : G.V) * Path G v w
  root v := ⟨v, .nil v⟩
  sub _ p := p.1
  shift _ p q := ⟨q.1, p.2.append q.2⟩
  d1 _ := rfl
  d4 _ _ _ := rfl
  d2 _ _ := rfl
  d3 _ p := congrArg (Sigma.mk p.1) (Path.append_nil p.2)
  d5 _ p q r := congrArg (Sigma.mk r.1)
                  (Path.append_assoc p.2 q.2 r.2)
```

Haskell.

```
-- A directed container (simplified: positions are just paths, not
-- endpoint-paired, since Haskell lacks dependent pairs in the same
   way).
data Network v e = Network
  { netRoot  :: v → (v, Path e)
  , netSub   :: v → (v, Path e) → v
  , netShift :: v → (v, Path e) → (v, Path e) → (v, Path e)
  }

network :: Digraph v e → Network v e
network g = Network
  { netRoot  = \v → (v, Nil)
  , netSub   = \_ p → fst p
  , netShift = \_ p q → (fst q, append (snd p) (snd q))
  }
```

Fidelity note. The five directed-container laws (D1–D5) encode the monoidal structure of path concatenation. D1 says the empty path ends at its own start node; D4 says the endpoint of a concatenated path is the endpoint of the second segment; D3 is right-unit; D5 is associativity. D2 is the left-unit law, but must be stated via a transport along D1 in the general directed-container formalism; here it reduces to `rfl` because D1 is definitional.

4 The Sawasdee Cellars wine network

4.1 Types and graph: `Wine.Facility`, `Wine.Leg`, `Wine.graph`

Definition 4.1 (Sawasdee Cellars network)**.** The **Sawasdee Cellars** shipping network (§2 of the paper) has:

- 11 facilities: three vineyards, the Sawasdee export agency, Melbourne Port, Thai Port, two distributors, three retail shops;
- 10 legs: three vineyard-to-Sawasdee legs (`v1s`, `v2s`, `v3s`), one Sawasdee-to-port (`smp`), the sea-freight leg (`sea`), two port-to-distributor legs (`td1`, `td2`), and three distributor-to-retail legs (`d1r1`, `d1r2`, `d2r3`).

This is a polytree (hence thin as a directed container).

Lean.

```
inductive Facility
  | vineyard1 | vineyard2 | vineyard3
```

```
  | sawasdee | melbournePort | thaiPort
  | distributor1 | distributor2
  | retail1 | retail2 | retail3
  deriving DecidableEq, Repr

inductive Leg
  | v1s | v2s | v3s | smp | sea
  | td1 | td2 | d1r1 | d1r2 | d2r3
  deriving DecidableEq, Repr

@[reducible] def graph : Digraph where
  V := Facility
  E := Leg
  src := Leg.src
  tgt := Leg.tgt
```

Haskell.

```
data Facility = Vineyard1 | Vineyard2 | Vineyard3
              | Sawasdee  | MelbournePort | ThaiPort
              | Distributor1 | Distributor2
              | Retail1 | Retail2 | Retail3

data Leg = V1s | V2s | V3s | Smp | Sea
         | Td1 | Td2 | D1r1 | D1r2 | D2r3
```

4.2 Definition: `Wine.network`

Definition 4.2 (Wine shipping network as a directed container)**.** `Wine.network` $:= \mathcal{N}(\text{graph})$ is the directed container obtained by applying Construction `con:network` to the Sawasdee Cellars graph. Its shapes are the 11 facilities, and its positions at a facility v are all directed routes out of v (including the trivial route).

Lean.

```
def network : DirectedContainer := Digraph.network graph
```

Haskell.

```
wineNetwork :: Network Facility Leg
wineNetwork = network wineGraph
```

4.3 Sanity checks (examples checked by `rfl`)

Proposition 4.3 (Route count at Vineyard_1)**.** The position set Pos(vineyard1) has exactly 9 elements: the empty route, plus one route to each of the eight facilities reachable from Vineyard_1 (Sawasdee, Melbourne Port, Thai Port, Distributor_1, Distributor_2, Retail_1, Retail_2, Retail_3).

Proposition 4.4 (Retail shop is a sink)**.** The position set Pos(retail1) has exactly 1 element: the empty route. Retail shops are sinks in the shipping graph.

Proposition 4.5 (`sub` is the route endpoint)**.** For the full $\text{Vineyard}_1 \to \text{Retail}_1$ route `fullRoute`: $\mathcal{N}(\text{graph}).\text{sub}(\text{vineyard1}, \text{fullRoute}) = \text{retail1}$.

Lean.

```
example : rVineyard1.length = 9 := rfl
example : rRetail1.length = 1 := rfl
example : network.sub vineyard1 fullRoute = retail1 := rfl
example : network.shift vineyard1 (network.root vineyard1) fullRoute
          = fullRoute := rfl
```

Haskell.

```
-- These are definitional equalities in Lean (rfl checks),
-- corresponding to properties of the concrete enumerations.
--
-- routeCount Vineyard1 = 9
-- routeCount Retail1   = 1
-- sub Vineyard1 fullRoute = Retail1
-- shift Vineyard1 (root Vineyard1) fullRoute = fullRoute
```

5 Translation conventions

1. **Dependent endpoint indices.** In Lean, `Path G v w` and positions $\Sigma_w \mathrm{Path}_G(v, w)$ are genuinely dependent types. In Haskell, we drop the endpoint tracking and use plain lists or pairs.
2. **Definitional vs. propositional equality.** Several directed- container laws hold definitionally (`rfl`) because the definitions are chosen so that the relevant type-level equalities reduce by computation. This precision is invisible in the Haskell reading.
3. **Proof bodies omitted.** Only definitions and theorem *statements* are translated. Lean checks the proofs; the human verifier checks that the *statements* match the intended mathematics.